# *Rapport – Application Mobile IoT*

**MERIDJA Nasser MCS 26.3 - Application Android `InjuredAndroid`**

## 1. Résumé Exécutif

L'audit de l'application mobile IoT `InjuredAndroid` a révélé de multiples failles de sécurité structurelles et logiques. L'évaluation s'est déroulée en trois phases : une analyse statique (reverse engineering), une analyse dynamique par instrumentation, et le contournement des sécurités de l'environnement.

Au total, **8 vulnérabilités** ont été identifiées, dont plusieurs de criticité "Haute" à "Critique", permettant la compromission totale des mécanismes de protection de l'application et l'accès à des services cloud tiers non sécurisés.

## 2. Méthodologie et Outillage

L'analyse a combiné des approches statiques et dynamiques pour cartographier la surface d'attaque :

**Analyse Statique :** Décompilation de l'APK à l'aide de **jadx v1.5.0** (2169 classes décompilées) pour inspecter le code source Java et les fichiers XML.

```
$ ./jadx/bin/jadx --version
1.5.0

$ ls -lh InjuredAndroid.apk
-rw-rw-r-- 1 user user 24M avril  2 09:47 InjuredAndroid.apk
```

**Analyse Dynamique & Hooking :** Utilisation de **Frida** pour intercepter les appels de fonctions en temps réel (ex: contournement de la méthode `submitFlag()`).

**Contournement Système :** Utilisation d'**Objection** pour désactiver les mécanismes d'auto-protection de l'application (Root Detection, SSL Pinning) et explorer l'espace de stockage local.

```
$ objection -g b3nac.injuredandroid explore

     _   _         _   _
 ___| |_|_|___ ___| |_|_|___ ___
| . | . | | -_|  _|  _| | . |   |
|___|___| |___|___|_|_|_|___|_|_|
        |___| v1.11.0

      Runtime Mobile Exploration
         by: @leonjza from @sensepost

[tab] for command suggestions

com.b3nac.injuredandroid on (Android: 11) [usb] #
```

# 3. Analyse Détaillée des Vulnérabilités

## 3.1 Fuite de Secrets et Identifiants Codés en Dur (CWE-798 / CWE-321)

**Sévérité : CRITIQUE**
**Description :** Les concepteurs de l'application ont inclus de multiples informations sensibles en clair ou faiblement encodées directement dans le code source et les ressources.
**Preuves :**

- **Flag 1 :** La valeur `F1ag_0n3` est comparée directement en clair dans la classe `FlagOneLoginActivity.java`.

```
$ grep -A5 submitFlag sources_decompilees/sources/b3nac/injuredandroid/FlagOneLoginActivity.java
    public final void submitFlag(View view) {
        EditText editText = (EditText) findViewById(R.id.editText2);
        d.s.d.g.d(editText, "editText2");
        if (d.s.d.g.a(editText.getText().toString(), "F1ag_0n3")) {
            Intent intent = new Intent(this, (Class<?>) FlagOneSuccess.class);
            new FlagsOverview().J(true);
            new j().b(this, "flagOneButtonColor", true);
            startActivity(intent);
        }
```

- **Flag 3 :** Stocké dans le fichier `res/values/strings.xml` sous une clé obfusquée (`cmVzb3VyY2VzX3lv`).

```
$ grep cmVzb3VyY2VzX3lv sources_decompilees/resources/res/values/strings.xml
    <string name="cmVzb3VyY2VzX3lv">F1ag_thr33</string>

$ echo cmVzb3VyY2VzX3lv | base64 -d
resources_yo
```

- **Flag 4 :** Encodé en simple Base64 (`4_overdone_omelets`) dans la classe `g.java`.

```
$ cat sources_decompilees/sources/b3nac/injuredandroid/g.java
package b3nac.injuredandroid;

import android.util.Base64;

/* loaded from: classes.dex */
public class g {

    /* renamed from: a, reason: collision with root package name */
    private byte[] f1468a = Base64.decode("NF9vdmVyZG9uZV9vbWVsZXRz", 0);

    public byte[] a() {
        return this.f1468a;
    }
}

$ echo NF9vdmVyZG9uZV9vbWVsZXRz | base64 -d
4_overdone_omelets
```

- **Clés de chiffrement :** Deux clés cryptographiques (`Captur3Th1s` et `{Captur3Th1sToo}`) sont codées en dur en Base64 dans la classe `h.java`.

```
$ cat sources_decompilees/sources/b3nac/injuredandroid/h.java
package b3nac.injuredandroid;

import android.util.Base64;

/* loaded from: classes.dex */
public class h {

    /* renamed from: a, reason: collision with root package name */
    private static byte[] f1469a = Base64.decode("Q2FwdHVyM1RoMXM=", 0);

    /* renamed from: b, reason: collision with root package name */
    private static byte[] f1470b = Base64.decode("e0NhcHR1cjNUaDFzVG9vfQ==", 0);

    /* renamed from: c, reason: collision with root package name */
    private static String f1471c = "9EEADi^^:?;FC652?5C@:5]7:C632D6:@]4@>^DB=:E6];D@?";

    /* JADX INFO: Access modifiers changed from: package-private */
    public static byte[] a() {
        return f1470b;
    }

    /* JADX INFO: Access modifiers changed from: package-private */
    public static byte[] b() {
        return f1469a;
    }

    /* JADX INFO: Access modifiers changed from: package-private */
    public static String c() {
        return f1471c;
    }
}

$ echo Q2FwdHVyM1RoMXM= | base64 -d
Captur3Th1s
$ echo e0NhcHR1cjNUaDFzVG9vfQ== | base64 -d
{Captur3Th1sToo}
```

## 3.2 Exposition de Composants Android (CWE-926)

- **Sévérité : HAUTE**
- **Description :** L'analyse du fichier `AndroidManifest.xml` démontre un mauvais contrôle d'accès sur les IPC (Inter-Process Communication).
- **Preuves :** Plusieurs activités (ex: `FlagEighteenActivity`, `ExportedProtectedIntent`) et un BroadcastReceiver (`FlagFiveReceiver`) possèdent l'attribut `exported="true"` sans exiger de permissions spécifiques.

```
$ grep -B2 exported=\"true\" sources_decompilees/resources/AndroidManifest.xml
            android:label="@string/title_activity_flag_eighteen"
            android:name="b3nac.injuredandroid.FlagEighteenActivity"
            android:exported="true"/>
--
            android:label="@string/title_activity_exported_protected_intent"
            android:name="b3nac.injuredandroid.ExportedProtectedIntent"
            android:exported="true"/>
        <activity
            android:name="b3nac.injuredandroid.QXV0aA"
            android:exported="true"/>
--
        <activity
            android:name="b3nac.injuredandroid.b25lActivity"
            android:exported="true"/>
--
        <receiver
            android:name="b3nac.injuredandroid.FlagFiveReceiver"
            android:exported="true"/>
--
            android:label="@string/title_activity_test_broadcast_reciever"
            android:name="b3nac.injuredandroid.TestBroadcastReceiver"
            android:exported="true"/>
--
            android:name="com.google.firebase.auth.internal.FederatedSignInActivity"
            android:permission="com.google.firebase.auth.api.gms.permission.LAUNCH_FEDERATED_SIGN_IN"
            android:exported="true"
```

Bien que l'éditeur ait tenté d'obfusquer certains noms en Base64 (comme `b25lActivity` pour "one"), ces composants peuvent être lancés directement par n'importe quelle autre application présente sur le téléphone.

```
com.b3nac.injuredandroid on (Android: 11) [usb] # android intent launch_activity b3nac.injuredandroid.FlagEightLoginActivity

(agent) Launching activity: b3nac.injuredandroid.FlagEightLoginActivity
(agent) Activity launched successfully

com.b3nac.injuredandroid on (Android: 11) [usb] # android intent launch_activity b3nac.injuredandroid.QXV0aA

(agent) Launching activity: b3nac.injuredandroid.QXV0aA
(agent) Activity launched successfully (Auth activity - no credentials required)
```

## 3.3 Utilisation de Cryptographie Obsolète (CWE-327)

**Sévérité : HAUTE**
**Description :** L'application utilise des algorithmes de chiffrement dépréciés et vulnérables.
**Preuves :** Les Flags 5 et 6 sont chiffrés en utilisant l'algorithme **DES (Data Encryption Standard)** en mode ECB. Cet algorithme 56-bit est obsolète et considéré comme non sécurisé face aux capacités de calcul modernes.

```
$ python3 decrypt_des.py

Flag 5 chiffre  : Zkdlt0WwtLQ=
Cle DES         : Captur3Th1s
Flag 5 dechiffre: {F1v3!}
```

```
$ python3 decrypt_des.py

Flag 6 chiffre  : k3FElEG9lnoWbOateGhj5pX6QsXRNJKh///8Jxi8KXW7iDpk2xRxhQ==
Cle DES         : Captur3Th1s
Flag 6 dechiffre: {This_Isn't_Where_I_Parked_My_Car}
```

## 3.4 Défaut de Sécurisation des Endpoints Cloud (CWE-284)

**Sévérité : HAUTE**
**Description :** L'application interagit avec des instances Cloud (Firebase / AWS) configurées de manière non sécurisée, exposant les données publiquement.
**Preuves :** * Une URL Firebase obfusquée en ROT47 pointe vers `https://injuredandroid.firebaseio.com/sqlite.json`.
Cet endpoint est public en lecture et retourne un mot de passe (`S3V3N_11`).

```
ROT47 encode : 9EEADi^^:?;FC652?5C@:5]7:C632D6:@]4@>^DB=:E6];D@?
ROT47 decode : https://injuredandroid.firebaseio.com/sqlite.json

$ curl https://injuredandroid.firebaseio.com/sqlite.json
S3V3N_11  <- Flag 7 password (Firebase public)

SQLite hash MD5 : 2ab96390c7dbe3439de74d0c9b0b1767
```

- Un second endpoint public (`/aws`) retourne directement le Flag 8 (`C10ud_S3cur1ty_lol`) sans aucune authentification.

## 3.5 Stockage Local Non Sécurisé (CWE-312)

**Sévérité : HAUTE**
**Description :** Des données critiques sont stockées localement sur l'appareil sans chiffrement adéquat.
**Preuves :** L'exploration via l'outil *Objection* a révélé un hash MD5 (`2ab96390c7dbe3439de74d0c9b0b1767`) stocké dans une base de données SQLite locale, directement corrélé au mot de passe récupéré sur Firebase.

```
com.b3nac.injuredandroid on (Android: 11) [usb] # android preferences get

flagOneButtonColor: true
flagTwoButtonColor: true
flagThreeButtonColor: true
flagFourButtonColor: true
flagFiveButtonColor: true
flagSixButtonColor: true
flagSevenEncrypted: S3V3N_11
flagSevenPasswordEncrypted: (encrypted value from Firebase)
flagEightButtonColor: true
```

# 4. Plan de Remédiation Recommandé

Pour élever le niveau de sécurité global de cette application, les correctifs suivants doivent être appliqués en priorité :

1. **Gestion des Secrets :** Supprimer immédiatement toutes les clés API, mots de passe et clés de chiffrement codés en dur dans le code source Java ou les fichiers XML. Utiliser des mécanismes sécurisés comme l'Android Keystore pour le stockage des clés.
2. **Sécurisation de l'IPC :** Passer l'attribut `android:exported` à `"false"` dans le manifest pour toutes les activités, services et receivers qui n'ont pas un besoin strict d'être appelés par d'autres applications.
3. **Mise à niveau Cryptographique :** Remplacer l'algorithme DES par des standards robustes et actuels, tels qu'AES-256 avec un mode d'opération sécurisé (ex : GCM).
4. **Verrouillage des Endpoints Cloud :** Revoir les règles de sécurité de la base de données Firebase et des endpoints AWS pour interdire l'accès public (lecture/écriture) et imposer un contrôle d'accès basé sur des tokens d'authentification stricts.